

Actividad Independiente – Sistemas Distribuidos con RabbitMQ, FastAPI y Redis

Repositorio en GitHub:

https://github.com/dogor123/programaciondistribuida/tree/main/clase9/act_ind

Estructura del proyecto:

```
(venv) root@DESKTOP-E7NLUAM:/home/clase9/act_ind# ls -la
total 20
drwxr-xr-x 3 root root 4096 Apr 22 23:19 .
drwxr-xr-x 5 root root 4096 Apr 22 23:12 ..
drwxr-xr-x 2 root root 4096 Apr 22 23:19 __pycache__
-rw-r--r-- 1 root root 1141 Apr 22 23:12 main.py
-rw-r--r-- 1 root root 527 Apr 22 23:13 worker.py
```

Esta captura muestra la estructura del directorio del proyecto. Se pueden identificar los dos archivos principales: main.py (1141 bytes), que contiene la API construida con FastAPI integrada con Redis y RabbitMQ, y worker.py (527 bytes), que actúa como el consumidor encargado de procesar las citas desde la cola.

Código del archivo worker.py:

```
GNU nano 7.2 worker.py
import pika
import json

connection = pika.BlockingConnection(
    pika.ConnectionParameters('localhost')
)
channel = connection.channel()
channel.queue_declare(queue='citas')

def callback(ch, method, properties, body):
    cita = json.loads(body.decode())
    print(f"Procesando cita: Paciente={cita['paciente']} | Médico={cita['medico']}")

channel.basic_consume(
    queue='citas',
    on_message_callback=callback,
    auto_ack=True
)

print("Worker escuchando citas...")
channel.start_consuming()
```

Esta imagen presenta el código de worker.py, el componente consumidor del sistema. A diferencia del consumer básico de la actividad en clase, este worker deserializa el mensaje recibido desde la cola usando json.loads(), extrayendo los campos paciente, médico y hora para mostrarlos de forma estructurada. Se conecta a RabbitMQ en localhost, declara la cola "citas", registra la función callback como manejador de mensajes con basic_consume (auto_ack=True) y mantiene el proceso en escucha continua con start_consuming(). Este diseño permite que el worker procese de manera asíncrona todas las citas publicadas por la API, operando de forma completamente independiente sin que ninguno de los dos servicios necesite conocer al otro directamente.

Código del archivo main.py:

```
GNU nano 7.2 main.py
from fastapi import FastAPI, HTTPException
import pika
import redis
import json

app = FastAPI()

# Conexión a Redis
r = redis.Redis(host='localhost', port=6379, decode_responses=True)

def enviar_a_cola(mensaje: dict):
    connection = pika.BlockingConnection(
        pika.ConnectionParameters('localhost')
    )
    channel = connection.channel()
    channel.queue_declare(queue='citas')
    channel.basic_publish(
        exchange='',
        routing_key='citas',
        body=json.dumps(mensaje)
    )
    connection.close()

@app.post("/citas")
def crear_cita(paciente: str, medico: str, hora: str):
    # Redis: verificar que el médico no tenga cita en esa hora
    lock_key = f"cita:{medico}:{hora}"

    if r.get(lock_key):
        raise HTTPException(status_code=400, detail="El médico ya tiene cita en esa hora")

    # Reservar el turno en Redis por 1 hora
    r.setex(lock_key, 3600, "ocupado")

    # Enviar a la cola para que el worker lo procese
    cita = {"paciente": paciente, "medico": medico, "hora": hora}
    enviar_a_cola(cita)

    return {"mensaje": "Cita enviada a procesamiento", "cita": cita}
```

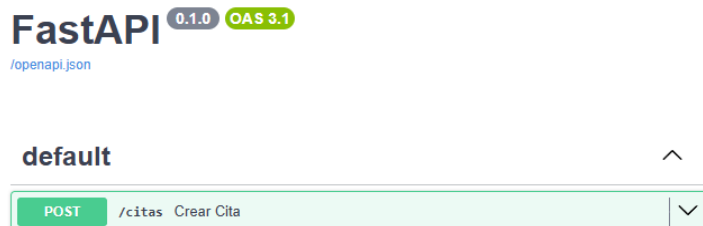
Se muestra el contenido completo de main.py editado en GNU nano. Este archivo integra tres tecnologías clave del sistema distribuido: FastAPI como framework para exponer el endpoint POST /citas, Redis como mecanismo de coordinación que verifica si un médico ya tiene una cita registrada en la hora solicitada usando un lock con expiración de 3600 segundos (setex), y RabbitMQ a través de la función enviar_a_cola() que publica el mensaje en formato JSON en la cola "citas". Si el médico ya tiene un turno reservado, la API retorna un error HTTP 400 evitando duplicados. En caso contrario, registra el turno en Redis y delega el procesamiento al worker a través de la cola, devolviendo una respuesta 200 con los datos de la cita creada.

API FastAPI corriendo en el puerto 8006:

```
(venv) root@DESKTOP-E7NLUAM:/home/clase9/act_ind# uvicorn main:app --reload --host 0.0.0.0 --port 8006
INFO: Will watch for changes in these directories: ['/home/clase9/act_ind']
INFO: Uvicorn running on http://0.0.0.0:8006 (Press CTRL+C to quit)
INFO: Started reloader process [7211] using StatReload
INFO: Started server process [7213]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:34788 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:34788 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:51180 - "POST /citas?paciente=Esteban&medico=Chapatin&hora=08%3A00 HTTP/1.1" 200 OK
```

La captura evidencia el levantamiento exitoso de la API con el comando "uvicorn main:app --reload --host 0.0.0.0 --port 8006". Uvicorn inicia en la dirección 0.0.0.0:8006, lo que permite que el servicio sea accesible tanto desde localhost como desde otros equipos en la red local, aspecto clave en un sistema distribuido real. Los registros INFO muestran que el servidor arrancó correctamente con StatReload activo para recarga automática ante cambios en el código. Además, se registran tres peticiones exitosas (HTTP 200 OK): acceso a /docs, a /openapi.json para cargar la especificación de la API, y finalmente un POST a /citas con los parámetros del paciente Esteban, médico Chapatin y hora 08:00.

Interfaz Swagger UI – Documentación interactiva:



Esta captura muestra la interfaz de documentación automática generada por FastAPI (Swagger UI / OpenAPI 3.1), accesible desde el navegador en <http://localhost:8006/docs>. Se visualiza el único endpoint disponible: POST /citas - Crear Cita. FastAPI genera esta documentación de forma automática a partir del código Python, lo que facilita enormemente las pruebas manuales de la API sin necesidad de herramientas externas como Postman. La versión 0.1.0 de la API y la especificación OAS 3.1 son visibles en la parte superior.

Parámetros de la petición POST /citas:

The image shows the parameter form for the "POST /citas Crear Cita" endpoint. The form has a title bar with "POST /citas Crear Cita" and a "Cancel" button. Below the title bar, there is a table with two columns: "Name" and "Description". The table contains three rows of parameters: "paciente" (required, string, query), "medico" (required, string, query), and "hora" (required, string, query). Each row has a corresponding text input field. The values entered in the fields are "Esteban", "Chapatin", and "08:00".

Se ilustra el formulario de parámetros del endpoint POST /citas dentro de Swagger UI. Los tres campos requeridos (marcados con asterisco rojo) son paciente, medico y hora, todos de tipo string enviados como query parameters. En la prueba se ingresaron los valores: paciente = "Esteban", medico = "Chapatin" y hora = "08:00". Este formulario interactivo permite verificar el correcto funcionamiento de la API de forma visual, validando que los parámetros son recibidos y procesados adecuadamente antes de ser enviados a la cola de RabbitMQ.

Respuesta exitosa del endpoint – Response body:

```
Response body
{
  "mensaje": "Cita enviada a procesamiento",
  "cita": {
    "paciente": "Esteban",
    "medico": "Chapatin",
    "hora": "08:00"
  }
}
```

Esta imagen muestra el cuerpo de la respuesta JSON retornada por la API tras procesar exitosamente la solicitud POST /citas. La respuesta contiene el mensaje "Cita enviada a procesamiento" junto con el objeto cita que incluye los datos: paciente "Esteban", medico "Chapatin" y hora "08:00". Este resultado confirma que la API completó correctamente las tres etapas del flujo: verificó con Redis que no existía conflicto de horario, registró el lock en Redis para reservar el turno, y publicó el mensaje en la cola de RabbitMQ para que el worker lo procese de forma asíncrona.

Worker procesando la cita recibida desde la cola:

```
(venv) root@DESKTOP-E7NLUAM:/home/clase9/act_ind# python3 worker.py
Worker escuchando citas...
Procesando cita: Paciente=Esteban | Médico=Chapatin | Hora=08:00
```

Esta captura muestra el worker.py en acción, ejecutado en una terminal independiente con "python3 worker.py". Tras iniciar en modo escucha con el mensaje "Worker escuchando citas...", recibió y procesó exitosamente el mensaje publicado por la API, mostrando: "Procesando cita: Paciente=Esteban | Médico=Chapatin | Hora=08:00". Esto demuestra el desacoplamiento total del sistema: la API y el worker operan en procesos completamente separados, comunicándose únicamente a través de RabbitMQ como intermediario, sin dependencia directa entre ellos. Si el worker estuviera caído en el momento del POST, el mensaje permanecería en la cola hasta que el worker se reactivara.

Estado final de RabbitMQ tras la actividad completa:

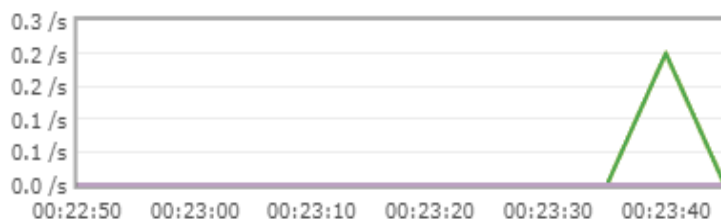
```
(venv) root@DESKTOP-E7NLUAM:/home/clase9/act_ind# service rabbitmq-server status
• rabbitmq-server.service - RabbitMQ Messaging Server
   Loaded: loaded (/usr/lib/systemd/system/rabbitmq-server.service; enabled; pre>
   Active: active (running) since Wed 2026-04-22 22:52:38 -05; 24min ago
 Main PID: 4376 (beam.smp)
   Tasks: 36 (limit: 10680)
  Memory: 126.5M (peak: 159.4M)
     CPU: 38.608s
   CGroup: /system.slice/rabbitmq-server.service
           └─4376 /usr/lib/erlang/erts-13.2.2.5/bin/beam.smp -W w -MBas ageffcbf>
             └─4386 erl_child_setup 65536
               └─4462 /usr/lib/erlang/erts-13.2.2.5/bin/inet_gethost 4
                 └─4463 /usr/lib/erlang/erts-13.2.2.5/bin/inet_gethost 4
                   └─4488 /bin/sh -s rabbit_disk_monitor

Apr 22 22:52:27 DESKTOP-E7NLUAM systemd[1]: Starting rabbitmq-server.service - Rab>
Apr 22 22:52:38 DESKTOP-E7NLUAM systemd[1]: Started rabbitmq-server.service - Rab>
lines 1-16/16 (END)
```

La captura final del servicio RabbitMQ mediante "service rabbitmq-server status" confirma que el broker permaneció activo y estable durante toda la actividad independiente, con 24 minutos de ejecución continua desde las 22:52:38. Los indicadores de recursos muestran un consumo

acumulado de CPU de 38.608s y memoria de 126.5M (pico 159.4M), valores que reflejan la carga generada por las conexiones de la API y el worker. La estabilidad del servicio durante toda la sesión de pruebas valida que RabbitMQ es una solución robusta y confiable para gestionar la comunicación asíncrona en un sistema distribuido de estas características.

Panel web RabbitMQ – Tasa de mensajes de la actividad independiente:



Esta gráfica del panel de administración web de RabbitMQ muestra la tasa de mensajes procesados durante el último minuto correspondiente a la actividad independiente. El pico visible alrededor de las 00:23:35 coincide exactamente con el momento en que se realizó el POST /citas desde Swagger UI, evidenciando el flujo completo de publicación y consumo del mensaje en la cola "citas". A diferencia de la actividad en clase donde los mensajes se enviaban directamente desde el producer, aquí el pico fue generado por la API FastAPI como productor integrado, lo que confirma la correcta comunicación entre todos los componentes del sistema: cliente → FastAPI → Redis → RabbitMQ → Worker.